

OBL1-OS

1. The process abstraction

1. Starting a process from a program on disk

What happens

When a user requests execution, the shell calls `fork()` to create a child process, and the child calls `execve()` with the path to the executable. This traps the CPU into kernel mode via the `syscall` instruction so the kernel can perform the following sequence of privileged operations. First, the kernel verifies the executable's format, which is typically ELF on Linux, failing with `ENOEXEC` otherwise. It then destroys the old address space inherited from `fork()` and builds a new one: a fresh page table is created, and the ELF program headers (`PT_LOAD`) describe how the segments (`.text` , `.data` , `.bss`) should be mapped - lazily via demand paging, so pages are only read from disk when first accessed. For dynamically linked binaries, the interpreter specified in the `PT_INTERP` segment (`ld-linux.so`) is mapped in to resolve shared libraries like `libc.so` at runtime. Next, the user stack is set up with `argc` , `argv[]` , `envp[]` , and the auxiliary vector, and the instruction pointer is initialized to the entry point - for dynamically linked binaries this is the interpreter's entry point, which eventually jumps to `_start` , which calls `main()` . Finally, the kernel's `syscall` return path executes a privileged return instruction (`sysret` or `iret` on x86-64, `eret` on ARM) that restores the saved user context and switches the CPU to user mode, and the program starts running, from then on competing for CPU time via the scheduler like any other process.

Why the kernel → user mode switch is necessary

The switch from kernel mode to user mode is necessary because the CPU has two privilege levels (on x86-64: ring 0 = kernel, ring 3 = user). In kernel mode all instructions are allowed, including privileged ones - direct hardware access, I/O, manipulating page tables (`CR3`), and changing the privilege level itself - while in user mode privileged instructions are forbidden, and any attempt raises a general protection fault.

This restriction exists for three reasons.

- **Protection:** if user programs could execute privileged instructions, a program could set the page-table base register itself and read or write the memory of other processes or the kernel, breaking

the isolation Linux enforces between processes.

- **Control:** running in user mode makes system calls the only way for a program to do anything privileged, so the kernel validates every request against permissions (UIDs, file permissions, capabilities) and remains in control of all shared resources.
- **Stability:** a buggy or malicious program is confined - it can only crash itself (e.g. segfault), not the operating system or other processes.

In short, the kernel sets everything up in kernel mode, then hands control to the program in user mode so it runs with restricted privileges while the Linux kernel retains full control of the machine.

2. Process memory and segments

1. The process' address space

Segment	Purpose
Stack ↓	Stores function call frames: local variables, function parameters, return addresses. Grows downward automatically on each function call and shrinks on return. Managed by the compiler/CPU (<code>push / pop</code> , <code>%rsp / %rbp</code>).
	<i>Unmapped address space</i>
Heap ↑	For dynamically allocated memory (<code>malloc / calloc / realloc</code> in C, <code>new</code> in C++). Grows upward, managed by the programmer - you must explicitly <code>free()</code> memory, otherwise it leaks.
Data	Holds global and static variables that are initialized to a non-zero value (e.g., <code>int var1 = 5; </code>). Sized and filled at compile time.
BSS (<i>data</i>)	Holds global/static variables that are uninitialized or initialized to zero (e.g., <code>int var2; </code>). Takes no space in the executable file - the kernel just zero-fills it at load time.
Text	The compiled machine code of the program (plus string literals/rodata). Read-only and shared between processes running the same program - if anyone tries to write to it, they get a segfault.
0x0	<i>Unmapped</i> - null-pointer guard

2. Why the `0x0` address is unavailable

Address `0x0` is deliberately kept unmapped by the OS for two main reasons:

1. Catch null-pointer bugs: In C, a pointer that is `NULL` has the value `0x0`. Since that page is not mapped, any read or write through a null pointer immediately triggers a segmentation fault instead of silently corrupting real memory. If address `0` were usable, a null-pointer dereference would read/write whatever happens to be there, and the bug could go unnoticed - or worse, be exploitable.
2. Convention / sentinel value: Many APIs use `0` to mean "no object/no address" (e.g., `malloc` returning `NULL` on failure). This only works reliably if address `0` can never be a valid, dereferenceable location.

So the kernel maps the first page as inaccessible precisely so that null-pointer dereferences fail loudly and early - turning a silent memory-corruption bug into an immediate, debuggable crash.

3. Variable types

	Global	Static	Local
Where it lives	Data segment (if initialized non-zero) or BSS (if zero/uninitialized)	Same as global, <i>but</i> a function-local <code>static</code> is hidden inside the function	Stack, inside the function's call frame
Lifetime	Entire program run	Entire program run	Only while the function is executing - destroyed on return
Scope	Entire program (all files can see it via <code>extern</code> ; in the same file it's visible from its declaration onward)	If declared outside a function: only that <code>.c</code> file (internal linkage). If declared inside a function: only that function, but it persists between calls	Only inside the block <code>{ }</code> where it's declared
Initial value	Zero-initialized if not initialized	Zero-initialized if not initialized	Garbage unless you initialize it yourself

	Global	Static	Local
Initialized when	Once, at program load (before main)	Once, at program load (before main)	Every time the declaration is executed

Key distinctions:

- **Global vs. static** (*file scope*): both live for the whole program in the data/BSS segments. The difference is linkage: a global can be accessed from other translation units via `extern`, while `static` restricts visibility to the current file - useful for encapsulation.
- **Local vs. static** (*function scope*): a `static` local keeps its value between function calls and is initialized only once, while a normal local is re-created and re-initialized, if you initialize it, on every call.
- **Local vs. the others**: locals are fast and automatically managed as they get freed on return, but they don't persist and can't be shared.

In `mem.c`, the following variables are as such:

```
#include <stdio.h>
#include <stdlib.h>

int var1 = 0;

void main() {
    int var2 = 1;
    int *var3 = (int *)malloc(sizeof(int)); // Note, since we are using malloc(), var3 will be a
                                           // pointer into the heap!
                                           // So the question is, where is the pointer stored?

    *var3 = 2;
    printf("Address: %x; Value: %d\n", &var1, var1);
    printf("Address: %x; Value: %d\n", &var2, var2);
    printf("Address: %x; Address: %x; Value: %d\n", &var3, var3, *var3);
}
```

- `var1` is global, initialized to `0`. Since it's zero, it therefore resides in the BSS segment. Lives for the whole program.
- `var2` is local and stored on the stack. Exists only while `main` runs; re-initialized to `1` on every call.
- `var3` - there are two things here:

- The pointer variable `var3` itself is local and stored on the stack.
- The memory it points to is on the heap, and lives until you `free()` it - even if `var3` goes out of scope.

3. Program code

1. Sizes of text, data, and bss segments

We can find the sizes of the text, data, and bss segments of the compiled `mem.c` program using the `size` command as such:

```
loginstud04:~/IDATT2202 Operativsystemer/Oppgave 01 - The process abstraction$ size mem
  text    data    bss     dec    hex filename
 1793     616      8    2417    971 mem
```

From this output, we can see that the segment sizes are as follows:

Segment	Size
text	1793 B
data	616 B
bss	8 B

2. The start address of the program

To find the start address of a program, we use the `objdump -f` command. When running for the compiled `mem.c`, we find that the start address of `mem` is at: `0x00000000000010a0`.

```
loginstud04:~/IDATT2202 Operativsystemer/Oppgave 01 - The process abstraction$ objdump -f mem

mem:      file format elf64-x86-64
architecture: i386:x86-64, flags 0x00000150:
HAS_SYMS, DYNAMIC, D_PAGED
start address 0x00000000000010a0
```

3. Disassembling the compiled program

To disassemble the compiled program we use `objdump -d mem`:

loginstud04:~/IDATT2202 Operativsystemer/Oppgave 01 - The process abstraction\$ objdump -d mem

mem: file format elf64-x86-64

Disassembly of section .init:

```
0000000000001000 <_init>:
1000: f3 0f 1e fa      endbr64
1004: 48 83 ec 08      sub    $0x8,%rsp
1008: 48 8b 05 d9 2f 00 00 mov    0x2fd9(%rip),%rax      # 3fe8 <__gmon_start__>
100f: 48 85 c0         test   %rax,%rax
1012: 74 02           je     1016 <_init+0x16>
1014: ff d0         call   *%rax
1016: 48 83 c4 08      add    $0x8,%rsp
101a: c3             ret
```

Disassembly of section .plt:

```
0000000000001020 <.plt>:
1020: ff 35 8a 2f 00 00 push   0x2f8a(%rip)          # 3fb0 <_GLOBAL_OFFSET_TABLE_>
1026: ff 25 8c 2f 00 00 jmp     *0x2f8c(%rip)        # 3fb8 <_GLOBAL_OFFSET_TABLE_>
102c: 0f 1f 40 00      nopl   0x0(%rax)
1030: f3 0f 1e fa      endbr64
1034: 68 00 00 00 00 push   $0x0
1039: e9 e2 ff ff ff   jmp     1020 <_init+0x20>
103e: 66 90           xchg    %ax,%ax
1040: f3 0f 1e fa      endbr64
1044: 68 01 00 00 00 push   $0x1
1049: e9 d2 ff ff ff   jmp     1020 <_init+0x20>
104e: 66 90           xchg    %ax,%ax
1050: f3 0f 1e fa      endbr64
1054: 68 02 00 00 00 push   $0x2
1059: e9 c2 ff ff ff   jmp     1020 <_init+0x20>
105e: 66 90           xchg    %ax,%ax
```

Disassembly of section .plt.got:

```
0000000000001060 <__cxa_finalize@plt>:
1060: f3 0f 1e fa      endbr64
1064: ff 25 8e 2f 00 00 jmp     *0x2f8e(%rip)        # 3ff8 <__cxa_finalize@GLIBC_2.2.5>
106a: 66 0f 1f 44 00 00 nopw    0x0(%rax,%rax,1)
```

Disassembly of section .plt.sec:

0000000000001070 <__stack_chk_fail@plt>:

```
1070:    f3 0f 1e fa                endbr64
1074:    ff 25 46 2f 00 00         jmp     *0x2f46(%rip)        # 3fc0 <__stack_chk_fail@GLIBC_2.2.5>
107a:    66 0f 1f 44 00 00         nopw   0x0(%rax,%rax,1)
```

0000000000001080 <printf@plt>:

```
1080:    f3 0f 1e fa                endbr64
1084:    ff 25 3e 2f 00 00         jmp     *0x2f3e(%rip)        # 3fc8 <printf@GLIBC_2.2.5>
108a:    66 0f 1f 44 00 00         nopw   0x0(%rax,%rax,1)
```

0000000000001090 <malloc@plt>:

```
1090:    f3 0f 1e fa                endbr64
1094:    ff 25 36 2f 00 00         jmp     *0x2f36(%rip)        # 3fd0 <malloc@GLIBC_2.2.5>
109a:    66 0f 1f 44 00 00         nopw   0x0(%rax,%rax,1)
```

Disassembly of section .text:

00000000000010a0 <_start>:

```
10a0:    f3 0f 1e fa                endbr64
10a4:    31 ed                      xor     %ebp,%ebp
10a6:    49 89 d1                   mov     %rdx,%r9
10a9:    5e                         pop     %rsi
10aa:    48 89 e2                   mov     %rsp,%rdx
10ad:    48 83 e4 f0                and     $0xfffffffffffffff0,%rsp
10b1:    50                         push    %rax
10b2:    54                         push    %rsp
10b3:    45 31 c0                   xor     %r8d,%r8d
10b6:    31 c9                      xor     %ecx,%ecx
10b8:    48 8d 3d ca 00 00 00       lea     0xca(%rip),%rdi        # 1189 <main>
10bf:    ff 15 13 2f 00 00         call    *0x2f13(%rip)        # 3fd8 <__libc_start_main@GLIBC_2.2.5>
10c5:    f4                         hlt
10c6:    66 2e 0f 1f 84 00 00       cs nopw 0x0(%rax,%rax,1)
10cd:    00 00 00
```

00000000000010d0 <deregister_tm_clones>:

```
10d0:    48 8d 3d 39 2f 00 00       lea     0x2f39(%rip),%rdi        # 4010 <__TMC_END__>
10d7:    48 8d 05 32 2f 00 00       lea     0x2f32(%rip),%rax        # 4010 <__TMC_END__>
10de:    48 39 f8                   cmp     %rdi,%rax
10e1:    74 15                      je      10f8 <deregister_tm_clones+0x28>
10e3:    48 8b 05 f6 2e 00 00       mov     0x2ef6(%rip),%rax        # 3fe0 <_ITM_deregister_tm_clones>
10ea:    48 85 c0                   test    %rax,%rax
```

```

10ed:    74 09                je     10f8 <deregister_tm_clones+0x28>
10ef:    ff e0                jmp     *%rax
10f1:    0f 1f 80 00 00 00 00 nopl   0x0(%rax)
10f8:    c3                    ret
10f9:    0f 1f 80 00 00 00 00 nopl   0x0(%rax)

```

0000000000001100 <register_tm_clones>:

```

1100:    48 8d 3d 09 2f 00 00 lea     0x2f09(%rip),%rdi        # 4010 <__TMC_END__>
1107:    48 8d 35 02 2f 00 00 lea     0x2f02(%rip),%rsi        # 4010 <__TMC_END__>
110e:    48 29 fe              sub     %rdi,%rsi
1111:    48 89 f0              mov     %rsi,%rax
1114:    48 c1 ee 3f          shr     $0x3f,%rsi
1118:    48 c1 f8 03          sar     $0x3,%rax
111c:    48 01 c6              add     %rax,%rsi
111f:    48 d1 fe              sar     $1,%rsi
1122:    74 14                je     1138 <register_tm_clones+0x38>
1124:    48 8b 05 c5 2e 00 00 mov     0x2ec5(%rip),%rax        # 3ff0 <_ITM_registerTM(
112b:    48 85 c0              test    %rax,%rax
112e:    74 08                je     1138 <register_tm_clones+0x38>
1130:    ff e0                jmp     *%rax
1132:    66 0f 1f 44 00 00    nopw   0x0(%rax,%rax,1)
1138:    c3                    ret
1139:    0f 1f 80 00 00 00 00 nopl   0x0(%rax)

```

0000000000001140 <__do_global_dtors_aux>:

```

1140:    f3 0f 1e fa          endbr64
1144:    80 3d c5 2e 00 00 00 cmpb    $0x0,0x2ec5(%rip)        # 4010 <__TMC_END__>
114b:    75 2b                jne     1178 <__do_global_dtors_aux+0x38>
114d:    55                    push    %rbp
114e:    48 83 3d a2 2e 00 00 cmpq    $0x0,0x2ea2(%rip)        # 3ff8 <__cxa_finalize@(
1155:    00
1156:    48 89 e5              mov     %rsp,%rbp
1159:    74 0c                je     1167 <__do_global_dtors_aux+0x27>
115b:    48 8b 3d a6 2e 00 00 mov     0x2ea6(%rip),%rdi        # 4008 <__dso_handle>
1162:    e8 f9 fe ff ff       call    1060 <__cxa_finalize@plt>
1167:    e8 64 ff ff ff       call    10d0 <deregister_tm_clones>
116c:    c6 05 9d 2e 00 00 01 movb    $0x1,0x2e9d(%rip)        # 4010 <__TMC_END__>
1173:    5d                    pop     %rbp
1174:    c3                    ret
1175:    0f 1f 00              nopl   (%rax)
1178:    c3                    ret
1179:    0f 1f 80 00 00 00 00 nopl   0x0(%rax)

```

0000000000001180 <frame_dummy>:

```
1180:    f3 0f 1e fa                endbr64
1184:    e9 77 ff ff ff            jmp     1100 <register_tm_clones>
```

0000000000001189 <main>:

```
1189:    f3 0f 1e fa                endbr64
118d:    55                          push    %rbp
118e:    48 89 e5                    mov     %rsp,%rbp
1191:    48 83 ec 20                  sub     $0x20,%rsp
1195:    64 48 8b 04 25 28 00        mov     %fs:0x28,%rax
119c:    00 00
119e:    48 89 45 f8                  mov     %rax,-0x8(%rbp)
11a2:    31 c0                        xor     %eax,%eax
11a4:    c7 45 ec 01 00 00 00        movl    $0x1,-0x14(%rbp)
11ab:    bf 04 00 00 00              mov     $0x4,%edi
11b0:    e8 db fe ff ff              call    1090 <malloc@plt>
11b5:    48 89 45 f0                  mov     %rax,-0x10(%rbp)
11b9:    48 8b 45 f0                  mov     -0x10(%rbp),%rax
11bd:    c7 00 02 00 00 00          movl    $0x2,(%rax)
11c3:    8b 05 4b 2e 00 00          mov     0x2e4b(%rip),%eax           # 4014 <var1>
11c9:    48 8d 35 44 2e 00 00        lea     0x2e44(%rip),%rsi           # 4014 <var1>
11d0:    48 8d 0d 31 0e 00 00        lea     0xe31(%rip),%rcx           # 2008 <_IO_stdin_used+0>
11d7:    89 c2                        mov     %eax,%edx
11d9:    48 89 cf                      mov     %rcx,%rdi
11dc:    b8 00 00 00 00              mov     $0x0,%eax
11e1:    e8 9a fe ff ff              call    1080 <printf@plt>
11e6:    8b 55 ec                      mov     -0x14(%rbp),%edx
11e9:    48 8d 45 ec                      lea     -0x14(%rbp),%rax
11ed:    48 8d 0d 14 0e 00 00        lea     0xe14(%rip),%rcx           # 2008 <_IO_stdin_used+0>
11f4:    48 89 c6                      mov     %rax,%rsi
11f7:    48 89 cf                      mov     %rcx,%rdi
11fa:    b8 00 00 00 00              mov     $0x0,%eax
11ff:    e8 7c fe ff ff              call    1080 <printf@plt>
1204:    48 8b 45 f0                  mov     -0x10(%rbp),%rax
1208:    8b 08                        mov     (%rax),%ecx
120a:    48 8b 55 f0                  mov     -0x10(%rbp),%rdx
120e:    48 8d 45 f0                  lea     -0x10(%rbp),%rax
1212:    48 8d 3d 07 0e 00 00        lea     0xe07(%rip),%rdi           # 2020 <_IO_stdin_used+0>
1219:    48 89 c6                      mov     %rax,%rsi
121c:    b8 00 00 00 00              mov     $0x0,%eax
1221:    e8 5a fe ff ff              call    1080 <printf@plt>
1226:    90                          nop
1227:    48 8b 45 f8                  mov     -0x8(%rbp),%rax
```

```

122b:      64 48 2b 04 25 28 00    sub    %fs:0x28,%rax
1232:      00 00
1234:      74 05                    je     123b <main+0xb2>
1236:      e8 35 fe ff ff        call   1070 <__stack_chk_fail@plt>
123b:      c9                    leave
123c:      c3                    ret

```

Disassembly of section .fini:

```

0000000000001240 <_fini>:
1240:      f3 0f 1e fa            endbr64
1244:      48 83 ec 08            sub    $0x8,%rsp
1248:      48 83 c4 08            add    $0x8,%rsp
124c:      c3                    ret

```

From the output, we can see that the function at the start address is `_start`. `_start` is the true entry point of the program - the very first code that runs when the OS loader (`execve`) starts the process. It is not `main`. It comes from the C runtime startup files (`crt1.o`) that the linker adds automatically.

4. Why the address changes between runs

When running the program several times, we can see that the addresses change between runs

```

loginstud04:~/IDATT2202 Operativsystemer/Oppgave 01 - The process abstraction$ ./mem
Address: bf036014; Value: 0
Address: d63eb90c; Value: 1
Address: d63eb910; Address: fa408010; Value: 2
loginstud04:~/IDATT2202 Operativsystemer/Oppgave 01 - The process abstraction$ ./mem
Address: dfa88014; Value: 0
Address: ced0d29c; Value: 1
Address: ced0d2a0; Address: 1f4f8010; Value: 2
loginstud04:~/IDATT2202 Operativsystemer/Oppgave 01 - The process abstraction$ ./mem
Address: eb8be014; Value: 0
Address: 871c1ac; Value: 1
Address: 871c1b0; Address: ff349010; Value: 2

```

This is due to ASLR (Address Space Layout Randomization) which is a security feature of the Linux kernel that randomizes where different memory regions are placed in the process's virtual address space each time a program runs.

4. The stack

2. Determine the default size of the stack for my Linux system

Using `ulimit --help`, we first see that `-s` is the appropriate flag to determine the default stack limit on my linux system. Then, after running `ulimit -s` we see that the default stack limit is set to 8192 KiB

```
loginstud04:~/IDATT2202 Operativsystemer/Oppgave 01 - The process abstraction$ ulimit -s
8192
```

Note

`ulimit -s` returns values in KiB units

3. Describe the observations and find the cause of the error

When we run the program, we get the following output:

```
main() frame address @ 0xe443d4f0
Segmentation fault      ./stackoverflow
```

We can deduce from the output that the program segfaults due to a stack overflow. This is because of the infinite recursion in `void func()`.

4. How the recursion count relates to the stack size

With the `./stackoverflow | grep func | wc -l` command, we first pipe the `stdout` to `grep` which compiles a list of all newlines containing `func`. Then we pipe the `grep` output to `wc` with the newline flag `-l` to count how many newlines contain the string `func` from the programs standard output.

Note

Since we print `func` twice each function call, we need to half the number returned by `wc -l`.

When running `./stackoverflow | grep func | wc -l` we get that the function was called $349003/2 \approx 174502$ times:

```
loginstud04:~/IDATT2202 Operativsystemer/Oppgave 01 - The process abstraction$ ./stackoverflow
349003
```

Note

The odd count of 349003 is explained by pipe buffering: since `stdout` is a pipe, glibc uses full buffering (~4 KiB blocks) instead of line buffering. When the program segfaults, the process dies immediately and the unflushed buffer is lost, so some complete lines never reach `grep`. The true call count is therefore slightly higher than 174502 - the odd remainder just means an odd number of buffered lines were lost, not that the crash happened between the two `printf()` calls.

To figure out this number's significance in relation to the 8 MiB limit, we first need to figure out the memory overhead for function calls in C compiled with GCC for my Ubuntu Linux system. We can do this by subtracting the address frame of frame $n + 1$ from frame n .

To do this, we use `./stackoverflow | grep "frame address" | head -3`, which greps the address frame output and stops the program after the first 3 lines as to not spam the console.

```
loginstud04:~/IDATT2202 Operativsystemer/Oppgave 01 - The process abstraction$ ./stackoverflow
main() frame address @ 0xd236feb0
func() frame address @ 0xd236fea0
func() frame address @ 0xd236fe70
```

We then use arithmetic to see that the difference in address frames is exactly 48 bytes

```
PS C:\Users\reale> 0xd236fea0 - 0xd236fe70
48
```

Now that we know that the total memory allocation per function call is 48 bytes, we multiply the number of calls with the amount of bytes allocated per call to see how much of the stack is used up before the program segfaults

```
PS C:\Users\reale> 174502 * 48
8376096
```

From this we can see why the 174502 recursion count is significant in relation to the 8 MiB default stack size. $174502 * 48 = 8376096$, which nearly fills the 8 MiB (8388608 bytes) stack - the

recursion consumed the entire stack allowed by `ulimit -s` , then touched the guard page and segfaulted

5. How much one recursive function call occupy in bytes

As explained in task 4.4, we can deduce that each recursive function call occupies 48 bytes